# Lab Sheet 4: SQL Operators

Solved queries with sample outputs — MySQL 8.x

**Assumptions:** The examples use sample tables `Student(StudentID, StudentName, Course, City, DepartmentID)` and `Employee(EmployeeID, EmployeeName, Salary, DepartmentName, ManagerID)`. Set operators such as INTERSECT and EXCEPT are not available in all MySQL versions; alternatives are shown where needed.

## Sample data summary

Students: Aarav (CSE, Dehradun), Diya (AIML, Delhi), Kabir (CSE, Dehradun), Meera (ECE, Roorkee), Ishita (AIML, NULL city). Employees: Rohan (Computer Science, 70000), Simran (AI & Data Science, 60000), Ankit (Computer Science, 65000), Priya (Electronics, 58000).

### *1. Perform addition of two numbers using +.*

**SQL query:**

```
SELECT 15 + 25 AS SumResult;
```

**Sample output / note:**

```
SumResult
40
```

### *2. Perform subtraction using -.*

**SQL query:**

```
SELECT 50 - 18 AS Difference;
```

**Sample output / note:**

```
Difference
32
```

### *3. Perform multiplication using *.*

**SQL query:**

```
SELECT 12 * 8 AS Product;
```

**Sample output / note:**

```
Product
96
```

### *4. Perform division using /.*

**SQL query:**

```
SELECT 45 / 5 AS Quotient;
```

**Sample output / note:**

```
Quotient
9.0000 (format may vary)
```

### *5. Calculate the total amount as Price × Quantity.*

**SQL query:**

```
SELECT ProductName, Price, Quantity, Price * Quantity AS TotalAmount
FROM OrderItems;
```

**Sample output / note:**

```
ProductName | Price | Quantity | TotalAmount
Notebook | 50.00 | 3 | 150.00
Pen | 10.00 | 5 | 50.00
Bag | 800.00 | 1 | 800.00
Requires the optional OrderItems sample table in the appendix.
```

### *6. Demonstrate AND with two conditions.*

**SQL query:**

```
SELECT StudentName, Course, City
FROM Student
WHERE Course = 'CSE' AND City = 'Dehradun';
```

**Sample output / note:**

```
StudentName | Course | City
Aarav | CSE | Dehradun
Kabir | CSE | Dehradun
```

### 7. Demonstrate OR with two conditions.

**SQL query:**

```
SELECT EmployeeName, DepartmentName
FROM Employee
WHERE DepartmentName = 'Computer Science' OR DepartmentName = 'Electronics';
```

**Sample output / note:**

```
EmployeeName | DepartmentName
Rohan | Computer Science
Ankit | Computer Science
Priya | Electronics
```

### 8. Demonstrate NOT with a condition.

**SQL query:**

```
SELECT StudentName, Course
FROM Student
WHERE NOT Course = 'CSE';
```

**Sample output / note:**

```
StudentName | Course
Diya | AIML
Meera | ECE
Ishita | AIML
```

### 9. Find students belonging to a specific course AND city.

**SQL query:**

```
SELECT StudentName
FROM Student
WHERE Course = 'AIML' AND City = 'Delhi';
```

**Sample output / note:**

```
StudentName
Diya
```

### 10. Find employees belonging to either of two departments.

**SQL query:**

```
SELECT EmployeeName, DepartmentName
FROM Employee
WHERE DepartmentName IN ('Computer Science', 'Electronics');
```

**Sample output / note:**

```
EmployeeName | DepartmentName
Rohan | Computer Science
Ankit | Computer Science
Priya | Electronics
```

### 11. Demonstrate the = operator.

**SQL query:**

```
SELECT StudentName, Course FROM Student WHERE Course = 'CSE';
```

**Sample output / note:**

```
StudentName | Course
Aarav | CSE
Kabir | CSE
```

### 12. Demonstrate the > operator.

**SQL query:**

```
SELECT EmployeeName, Salary FROM Employee WHERE Salary > 60000;
```

**Sample output / note:**

```
EmployeeName | Salary
Rohan | 70000.00
Ankit | 65000.00
```

### 13. Demonstrate the < operator.

**SQL query:**

```
SELECT EmployeeName, Salary FROM Employee WHERE Salary < 60000;
```

**Sample output / note:**

```
EmployeeName | Salary
Priya | 58000.00
```

### 14. Demonstrate >= and <=.

**SQL query:**

```
SELECT EmployeeName, Salary FROM Employee WHERE Salary >= 60000 AND Salary <= 70000;
```

**Sample output / note:**

```
EmployeeName | Salary
Rohan | 70000.00
Simran | 60000.00
Ankit | 65000.00
```

### 15. Demonstrate the <>/!= operator.

**SQL query:**

```
SELECT StudentName, Course FROM Student WHERE Course <> 'CSE';
```

**Sample output / note:**

```
StudentName | Course
Diya | AIML
Meera | ECE
Ishita | AIML
Note: <> and != both mean 'not equal' in MySQL.
```

### 16. Demonstrate BETWEEN for salary range.

**SQL query:**

```
SELECT EmployeeName, Salary FROM Employee WHERE Salary BETWEEN 60000 AND 70000;
```

**Sample output / note:**

```
EmployeeName | Salary
Rohan | 70000.00
Simran | 60000.00
Ankit | 65000.00
BETWEEN includes both endpoints.
```

### 17. Demonstrate IN for selecting multiple cities.

**SQL query:**

```
SELECT StudentName, City FROM Student WHERE City IN ('Dehradun', 'Roorkee');
```

**Sample output / note:**

```
StudentName | City
Aarav | Dehradun
Kabir | Dehradun
Meera | Roorkee
```

### 18. Demonstrate NOT IN.

**SQL query:**

```
SELECT StudentName, City FROM Student WHERE City NOT IN ('Dehradun', 'Roorkee');
```

**Sample output / note:**

```
StudentName | City
Diya | Delhi
NULL cities are not returned by this condition.
```

### 19. Demonstrate LIKE for names beginning with a particular letter.

**SQL query:**

```
SELECT StudentName FROM Student WHERE StudentName LIKE 'A%';
```

**Sample output / note:**

```
StudentName
Aarav
```

### 20. Demonstrate IS NULL and IS NOT NULL.

**SQL query:**

```
SELECT StudentName, City FROM Student WHERE City IS NULL;

SELECT StudentName, City FROM Student WHERE City IS NOT NULL;
```

**Sample output / note:**

```
First query (IS NULL):
Ishita | NULL

Second query (IS NOT NULL):
Aarav | Dehradun
Diya | Delhi
Kabir | Dehradun
Meera | Roorkee
```

### 21. Demonstrate UNION using two compatible queries.

**SQL query:**

```
SELECT StudentName AS PersonName FROM Student WHERE Course = 'CSE'
UNION
SELECT EmployeeName AS PersonName FROM Employee WHERE Salary >= 65000;
```

**Sample output / note:**

```
PersonName
Aarav
Ankit
Kabir
Rohan
UNION removes duplicate rows.
```

### 22. Demonstrate UNION ALL.

**SQL query:**

```
SELECT StudentName AS PersonName FROM Student WHERE Course = 'CSE'
UNION ALL
SELECT EmployeeName AS PersonName FROM Employee WHERE Salary >= 65000;
```

**Sample output / note:**

```
PersonName
Aarav
Kabir
Rohan
Ankit
UNION ALL retains duplicate rows (if any).
```

### 23. Demonstrate INTERSECT where supported by the DBMS.

**SQL query:**

```
SELECT City AS Place FROM Student WHERE City IS NOT NULL
INTERSECT
SELECT 'Dehradun' AS Place;
```

**Sample output / note:**

```
Place
Dehradun
MySQL versions before 8.0.31 do not support INTERSECT. A compatible alternative is:
SELECT DISTINCT City AS Place FROM Student WHERE City = 'Dehradun';
```

### 24. Demonstrate EXCEPT/MINUS where supported.

**SQL query:**

```
SELECT City AS Place FROM Student WHERE City IS NOT NULL
EXCEPT
SELECT 'Dehradun' AS Place;
```

**Sample output / note:**

```
Place
Delhi
Roorkee
MySQL does not support EXCEPT in common versions. Alternative:
SELECT DISTINCT City AS Place FROM Student
WHERE City IS NOT NULL AND City <> 'Dehradun';
Oracle uses MINUS instead of EXCEPT.
```

### 25. Use arithmetic, logical, comparison and special operators together.

**SQL query:**

```
SELECT StudentName, Course, City
FROM Student
WHERE Course IN ('CSE', 'AIML')
AND City IS NOT NULL
AND StudentID > 0
AND StudentName LIKE 'A%';
```

**Sample output / note:**

```
StudentName | Course | City
Aarav | CSE | Dehradun
This combines IN, AND, IS NOT NULL, > and LIKE. Arithmetic example: StudentID * 10 AS IDValue.
```

# Appendix: Sample tables and data

Run the following SQL once in MySQL Workbench before running the queries. The set-operator syntax may need adapting to your DBMS/version.

```
CREATE TABLE Student (
StudentID INT PRIMARY KEY,
StudentName VARCHAR(50),
Course VARCHAR(20),
City VARCHAR(50),
DepartmentID INT NULL
);
INSERT INTO Student VALUES
(1, 'Aarav', 'CSE', 'Dehradun', 1),
(2, 'Diya', 'AIML', 'Delhi', 2),
(3, 'Kabir', 'CSE', 'Dehradun', 1),
(4, 'Meera', 'ECE', 'Roorkee', 3),
(5, 'Ishita', 'AIML', NULL, 2);

CREATE TABLE Employee (
EmployeeID INT PRIMARY KEY,
EmployeeName VARCHAR(50),
Salary DECIMAL(10,2),
DepartmentName VARCHAR(60),
ManagerID INT NULL
);
INSERT INTO Employee VALUES
(201, 'Rohan', 70000, 'Computer Science', NULL),
(202, 'Simran', 60000, 'AI & Data Science', 201),
(203, 'Ankit', 65000, 'Computer Science', 201),
(204, 'Priya', 58000, 'Electronics', 202);

CREATE TABLE OrderItems (
ItemID INT PRIMARY KEY,
ProductName VARCHAR(50),
Price DECIMAL(10,2),
Quantity INT
);
INSERT INTO OrderItems VALUES
(1, 'Notebook', 50, 3),
(2, 'Pen', 10, 5),
(3, 'Bag', 800, 1);
```